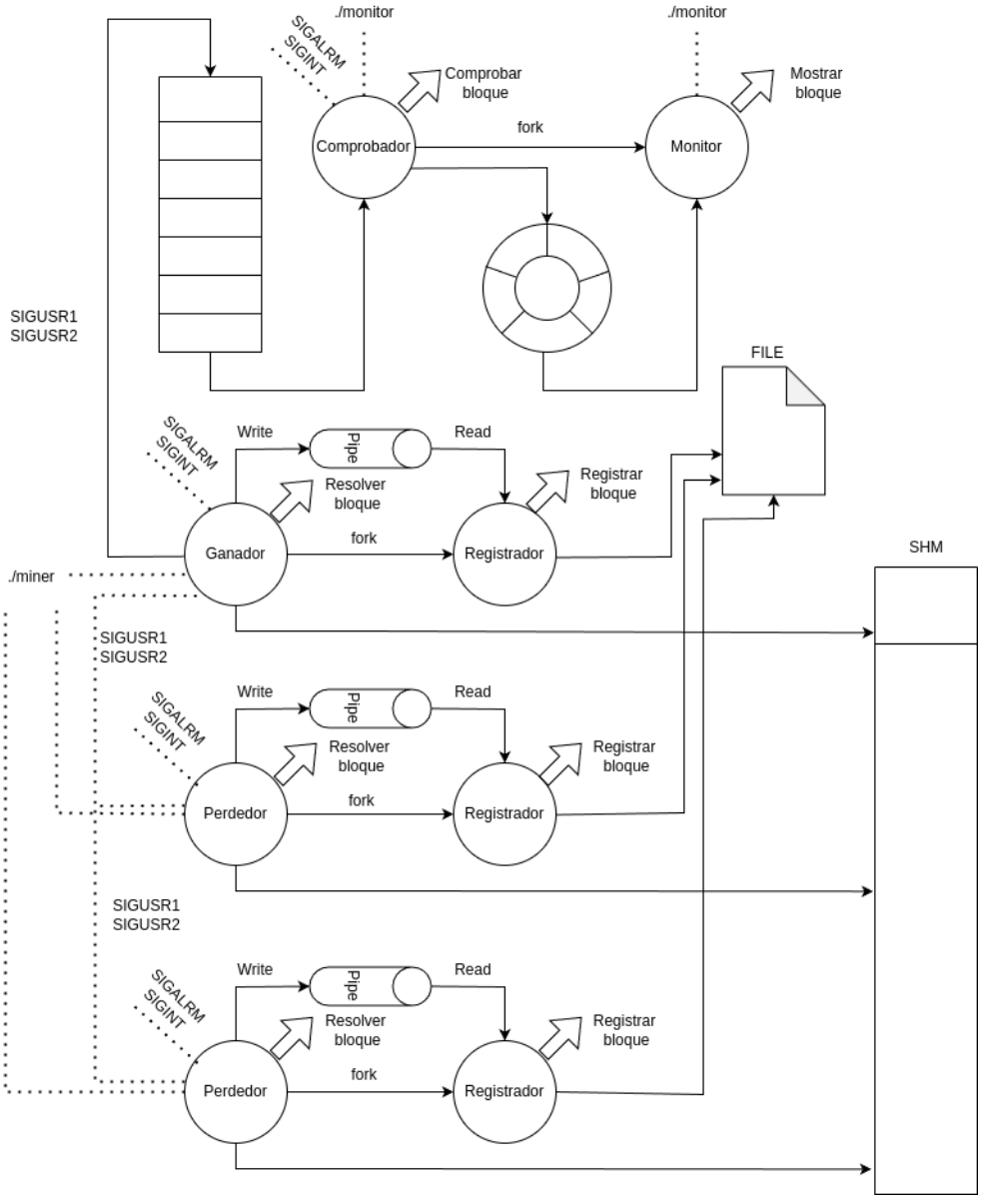
Entregables y ejemplos de ejecución

a) Realizar un diagrama que muestre el diseño del sistema, incluyendo los distintos componentes (procesos) y sus jerarquías, así como los mecanismos de comunicación y sincronización entre ellos.



b) Describir el diseño del sistema, incluyendo los aspectos más relevantes, y los principales problemas que se han encontrado durante la implementación y cómo se han abordado.

El proceso de arranque de miner está diseñado para validar desde el primer momento los parámetros de ejecución, asegurando que el número de segundos de vida del proceso sea al

menos uno y que el número de hilos solicitados se encuentre entre uno y el máximo permitido. Si los argumentos no cumplen estas condiciones, el programa informa al usuario y termina inmediatamente, evitando comportamientos indeseados más adelante.

```c
if (argc != 3){
    printf("Formato: %s <N_SECONDS> <N_THREADS>\n", argv[0]);
    exit(EXIT_FAILURE);
}

n_seconds = atoi(argv[1]);
if (n_seconds < 1){
    printf("El numero de segundos tiene que ser mayor o igual a 1\n");
    exit(EXIT_FAILURE);
}

n_threads = atoi(argv[2]);
if (n_threads > MAX_THREADS || n_threads < 1){
    printf("El numero de threads tiene que ser mayor o igual a 1 y menor o igual a %d\n", MAX_THREADS);
    exit(EXIT_FAILURE);
}
```

Para mantener un registro opcional de los bloques resueltos, el proceso principal crea un pipe antes de bifurcarse con fork(). El padre retiene el extremo de escritura y se encarga de toda la lógica de minería, mientras que el hijo, el “Registrador”, cierra el extremo de escritura, abre (o crea) un archivo exclusivo por PID y se queda leyendo bloques del pipe, formateándolos y escribiéndolos en disco. De este modo, la tarea de registro queda separada de la lógica crítica de minería.

```c
/* Se hace fork */
pid = fork();
if (pid == -1) {
    perror("fork");
    exit(EXIT_FAILURE);

}
if (pid) { /* Miner */
    pid = getpid();

    /* Cierra el extremo de lectura */
    close(fd_pipe[0]);

    mining_process(pid, n_threads, n_seconds, fd_pipe[1]);

    wait(&state);
    if (WEXITSTATUS(state)) {
        printf("Registrator failed\n");
    }

    /* Cierra el extremo de escritura */
    close(fd_pipe[1]);

    exit(EXIT_SUCCESS);

} else { /* El proceso Registrador arranca como hijo de un proceso Minero. */

    /* Se cierra el descriptor de escritura de */
    close(fd_pipe[1]);

    /* Se construye el nombre del archivo con */
    snprintf(file, sizeof(file), "%s_%d.dat", REG_FILE, getppid());

    /* Se intenta crear y abrir el archivo de forma exclusiva */
    fd = open(file, O_WRONLY | O_APPEND | O_CREAT | O_EXCL, 0666);
    if (fd == -1) {
        if (errno == EEXIST) {
            /* Si ya existe, se abre el archivo sin el flag EXCL */
            fd = open(file, O_WRONLY | O_APPEND | O_CREAT, 0666);
            if (fd == -1) {
                perror("Error al abrir el archivo");
                exit(EXIT_FAILURE);
            }
        } else {
            perror("Error al crear el archivo");
            exit(EXIT_FAILURE);
```

```
#define REG_FILE "registrator"
Expands to:
"registrator"
```

```
    }
}

/* Registro de bloque: Mientras la tubería que lo conecta con Minero se mantenga abierta, lee los
   bloques, y los escribe usando la función dprintf en un fichero cuyo nombre dependa del PID de
   su proceso padre.*/
while (1) {
    i = 0;
    fd_read = read(fd_pipe[0], &block, sizeof(block));
    if (fd_read == -1) {
        perror("Error en read desde el pipe");
        exit(EXIT_FAILURE);
    }
    /* Si no se leyeron datos o se recibió un bloque terminador, se sale del bucle */
    if (fd_read == 0 || block.solution == -1) {
        break;
    }

    /* Escribe la cabecera del bloque */
    if (block.posVotes >= block.totalVotes / 2) {
        dprintf(fd, "Id : %03d\nWinner : %d\nTarget : %08ld", j, block.winner, block.target);
        dprintf(fd, "\nSolution : %08ld (validated)\nVotes : %d/%d\nWallets :", block.solution, block.posVotes, block.totalVotes);
    } else {
        dprintf(fd, "Id : %03d\nWinner : %d\nTarget : %08ld", j, block.winner, block.target);
        dprintf(fd, "\nSolution : %08ld (rejected)\nVotes : %d/%d\nWallets :", block.solution, block.posVotes, block.totalVotes);
    }

    /* Imprime la lista de wallets */
    while (i < block.size) {
        dprintf(fd, " %d:%d", block.wallets[i].pid, block.wallets[i].monedas);
        i++;
    }
    dprintf(fd, "\n\n");
    j++;
}

/* Se cierra el archivo y el extremo de lectura del pipe */
close(fd);
close(fd_pipe[0]);
exit(EXIT_SUCCESS);
}
```

La coordinación entre procesos se basa en señales UNIX gestionadas con sigaction. Cada señal, SIGUSR1 para iniciar rondas, SIGUSR2 para señalar que hay solución y empezar votación, SIGALRM para el fin de tiempo y SIGINT para terminar, únicamente levanta una bandera atómica. Esta aproximación simplifica el manejo de interrupciones sin correr el riesgo de ejecutar código complejo dentro de los manejadores de señal.

```c
static volatile sig_atomic_t  got_usr1 = 0, got_usr2 = 0, got_alrm = 0, got_int = 0;

void handler(int sig){
    if (sig == SIGUSR1){
        got_usr1 = 1; /* Señal de inicio de ronda */
    } else if (sig == SIGUSR2){
        got_usr2 = 1; /* Señal de solución encontrada y comienzo de votación */
    } else if (sig == SIGALRM){
        got_alrm = 1; /* Señal de fin de tiempo de ejecución */
    } else if (sig == SIGINT){
        got_int  = 1; /* Señal de interrupción */
    }
}
```

Al arrancar, el primer minero crea y dimensiona un objeto de memoria compartida llamado SHM_MINER; tras truncarlo al tamaño de la estructura de datos, lo mapea en su espacio de direcciones e inicializa todos los campos a cero mediante una función dedicada.

```c
/* Conexión a la memoria compartida */
fd_shm = shm_open(SHM_MINER, O_RDWR | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR);
if (fd_shm == -1) {
    if (errno == EEXIST) {
        if ((fd_shm = shm_open(SHM_MINER, O_RDWR, S_IRUSR | S_IWUSR)) == -1) {
            perror("Error opening the shared memory segment");
            exit(EXIT_FAILURE);
        }
        else {
            perror("Shared memory segment open\n");
            flag ++;
        }
    } else {
        perror("Error creating the shared memory segment\n");
        exit(EXIT_FAILURE);
    }
} else {
    perror("Shared memory segment created\n");
    if (ftruncate (fd_shm , sizeof(MiningData)) == -1) {
        perror("ftruncate");
        close(fd_shm);
        exit(EXIT_FAILURE);
    }
}
```

A continuación, fija el objetivo inicial de la ronda en cero y añade su propio PID al listado de wallets, señalando a todos (incluido él mismo) con SIGUSR1 para comenzar la primera ronda. Los mineros posteriores, al detectar que el objeto ya existe, simplemente lo abren, mapean y esperan pasivamente hasta que el primer minero complete la inicialización, tras lo cual añaden su PID y aguardan la señal de arranque.

```c
/* Registro de minero y arranque de primera ronda*/
if (!flag){
    buffer = initMinerData(buffer);
    if (buffer == NULL){
        exit(EXIT_FAILURE);
    }

    buffer->target = 0;

    /* Se añade primer minero */
    if (addMiner(buffer, self)){
        printf("El sistem no acepta mas mineros\n");
        exit(EXIT_SUCCESS);
    }

    /* Notifica con SIGUSR1 a todos para arrancar ronda */
    for (i = 0; i < buffer->size; i++){
        if(kill(buffer->wallets[i].pid, SIGUSR1) == -1){
            perror("kill");
            exit(EXIT_FAILURE);
        }
    }
} else{
    /* Se añade minero adicional*/
    if (addMiner(buffer, getpid())){
        printf("El sistem no acepta mas mineros\n");
        exit(EXIT_SUCCESS);
    }
}

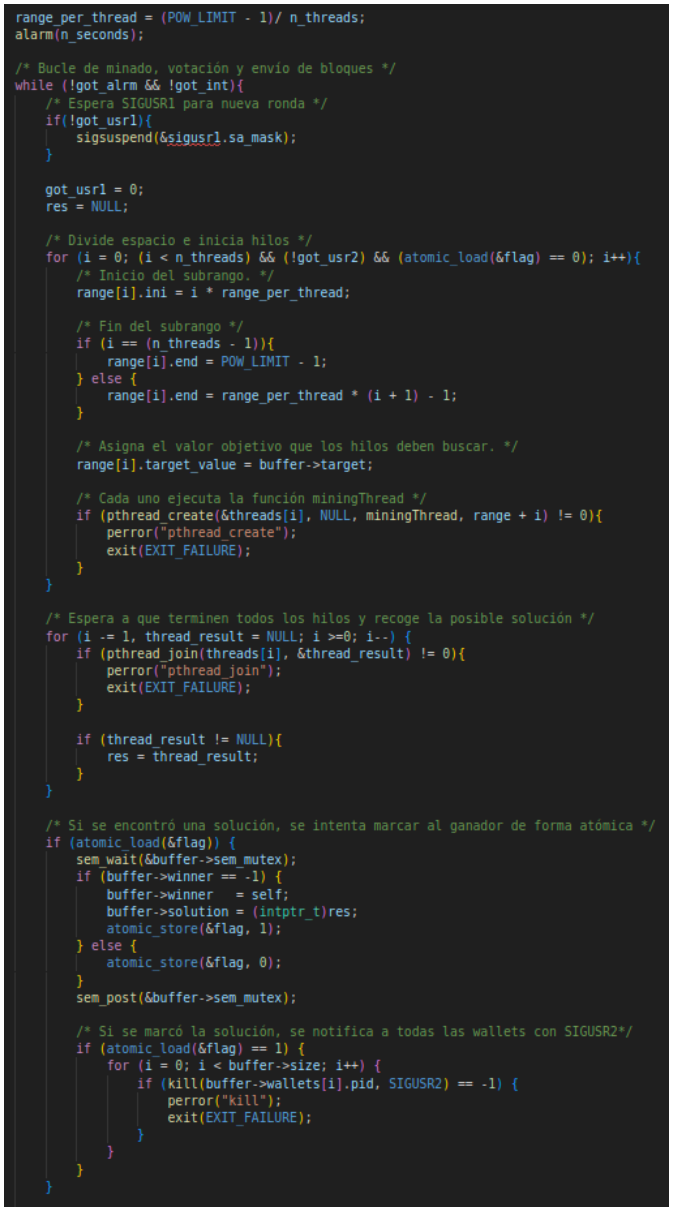
/* Se abre cola de mensajes y semáforo para comunicación con monitor */
if ((mq = mq_open(MQ_NAME,O_CREAT |O_WRONLY | O_EXCL, S_IRUSR | S_IWUSR, &attributes)) == (mqd_t)-1) {
    if (errno == EEXIST){
        if ((mq = mq_open(MQ_NAME, O_WRONLY, S_IRUSR | S_IWUSR, &attributes)) == -1) {
            perror("mq_open");
            exit(EXIT_FAILURE);
        }

    }
    else{
        perror("mq_open");
        exit(EXIT_FAILURE);
    }
}

if (( sem = sem_open ( SEM_NAME , O_RDWR | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR , 0)) == SEM_FAILED ) {
    if (errno == EEXIST){
        if ((sem = sem_open ( SEM_NAME , O_RDWR, S_IRUSR | S_IWUSR , 0)) == SEM_FAILED) {
            perror("sem_open(EEXIST)");
            exit(EXIT_FAILURE);
        }
    }
    else{
        perror("sem_open");
        exit(EXIT_FAILURE);
    }
}
```

Cada ronda de minería se desarrolla dentro de un bucle que se repite hasta que salte el temporizador o llegue una interrupción externa. El proceso espera la señal de inicio, divide el espacio de búsqueda entre los hilos especificados y lanza cada uno con pthread_create. Al hacer pthread_join recopila cualquier resultado, y si algún hilo local o un compañero de red indica con SIGUSR2 que ha hallado la solución, se establece una variable atómica y se

coordina con un semáforo para designar de forma segura al ganador de la ronda y notificar a todos con SIGUSR2.



```c
range_per_thread = (POW_LIMIT - 1)/ n_threads;
alarm(n_seconds);

/* Bucle de minado, votación y envío de bloques */
while (!got_alrm && !got_int){
    /* Espera SIGUSR1 para nueva ronda */
    if(!got_usr1){
        sigsuspend(&sigusr1.sa_mask);
    }

    got_usr1 = 0;
    res = NULL;

    /* Divide espacio e inicia hilos */
    for (i = 0; (i < n_threads) && (!got_usr2) && (atomic_load(&flag) == 0); i++){
        /* Inicio del subrango. */
        range[i].ini = i * range_per_thread;

        /* Fin del subrango */
        if (i == (n_threads - 1)){
            range[i].end = POW_LIMIT - 1;
        } else {
            range[i].end = range_per_thread * (i + 1) - 1;
        }

        /* Asigna el valor objetivo que los hilos deben buscar. */
        range[i].target_value = buffer->target;

        /* Cada uno ejecuta la función miningThread */
        if (pthread_create(&threads[i], NULL, miningThread, range + i) != 0){
            perror("pthread_create");
            exit(EXIT_FAILURE);
        }
    }

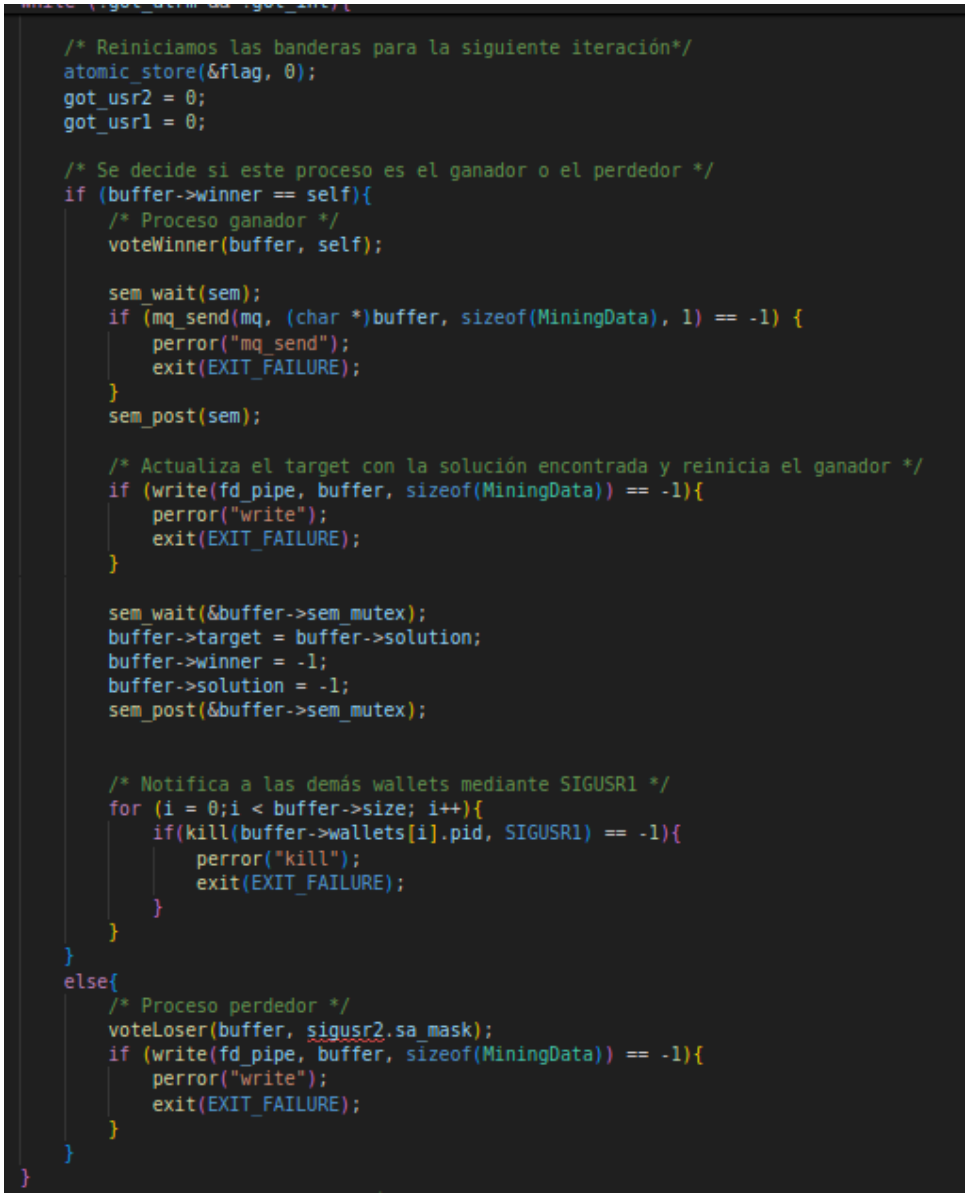
    /* Espera a que terminen todos los hilos y recoge la posible solución */
    for (i -= 1, thread_result = NULL; i >=0; i--) {
        if (pthread_join(threads[i], &thread_result) != 0){
            perror("pthread_join");
            exit(EXIT_FAILURE);
        }

        if (thread_result != NULL){
            res = thread_result;
        }
    }

    /* Si se encontró una solución, se intenta marcar al ganador de forma atómica */
    if (atomic_load(&flag)) {
        sem_wait(&buffer->sem_mutex);
        if (buffer->winner == -1) {
            buffer->winner   = self;
            buffer->solution = (intptr_t)res;
            atomic_store(&flag, 1);
        } else {
            atomic_store(&flag, 0);
        }
        sem_post(&buffer->sem_mutex);

        /* Si se marcó la solución, se notifica a todas las wallets con SIGUSR2*/
        if (atomic_load(&flag) == 1) {
            for (i = 0; i < buffer->size; i++) {
                if (kill(buffer->wallets[i].pid, SIGUSR2) == -1) {
                    perror("kill");
                    exit(EXIT_FAILURE);
                }
            }
        }
    }
```

La fase de proclamación del ganador y la votación se implementa con semáforos y ciclos de espera ligera. El hilo principal, tras detectar que la bandera atómica indica solución, entra en la región crítica para asignarse el rol de winner y almacenar el valor de la solución. De inmediato dispara SIGUSR2 a todos los mineros, momento en el cual cada uno comprueba la validez de la solución y actualiza los contadores de votos en memoria compartida (positivos y totales) bajo protección de semáforo. El ganador espera de forma no activa a que todos voten o caduque un límite de intentos; si consigue mayoría, recibe una moneda adicional en su wallet.



```
    /* Reiniciamos las banderas para la siguiente iteración*/
    atomic_store(&flag, 0);
    got_usr2 = 0;
    got_usr1 = 0;

    /* Se decide si este proceso es el ganador o el perdedor */
    if (buffer->winner == self){
        /* Proceso ganador */
        voteWinner(buffer, self);

        sem_wait(sem);
        if (mq_send(mq, (char *)buffer, sizeof(MiningData), 1) == -1) {
            perror("mq_send");
            exit(EXIT_FAILURE);
        }
        sem_post(sem);

        /* Actualiza el target con la solución encontrada y reinicia el ganador */
        if (write(fd_pipe, buffer, sizeof(MiningData)) == -1){
            perror("write");
            exit(EXIT_FAILURE);
        }

        sem_wait(&buffer->sem_mutex);
        buffer->target = buffer->solution;
        buffer->winner = -1;
        buffer->solution = -1;
        sem_post(&buffer->sem_mutex);


        /* Notifica a las demás wallets mediante SIGUSR1 */
        for (i = 0;i < buffer->size; i++){
            if(kill(buffer->wallets[i].pid, SIGUSR1) == -1){
                perror("kill");
                exit(EXIT_FAILURE);
            }
        }
    }
    else{
        /* Proceso perdedor */
        voteLoser(buffer, sigusr2.sa_mask);
        if (write(fd_pipe, buffer, sizeof(MiningData)) == -1){
            perror("write");
            exit(EXIT_FAILURE);
        }
    }
}
```

Para notificar al monitor central, el ganador envía el bloque resuelto por cola de mensajes (mq_send) y actualiza el target en la memoria compartida con la solución obtenida. Tras ello restablece los campos de ganador y solución a valores nulos y dispara SIGUSR1, señal inequívoca de que comienza una nueva ronda de minería.

```c
/* Se guarda el estado final según el tamaño previo a eliminar el minero. */
last = buffer->size == 1;

/* Se elimina al minero */
removeMiner(buffer, self);

if (last){ /* Ultimo minero */

    buffer->target = -1;
    buffer->solution = -1;

    if (write(fd_pipe, buffer, sizeof(MiningData)) == -1){
        perror("write");
        exit(EXIT_FAILURE);
    }

    sem_wait(sem);
    if (mq_send(mq, (char *)buffer, MSG_SIZE, 1) == -1) {
        perror("mq_send");
        exit(EXIT_FAILURE);
    }
    sem_post(sem);

    munmap(buffer, sizeof(MiningData));
    sem_unlink(SEM_NAME);
    shm_unlink(SHM_MINER);
    mq_close(mq);
    sem_close(sem);

} else { /* No es el último minero. */
    /* Si el minero actual es el ganador, se envia SIGUSR1 a todas las wallets.*/
    if (buffer->winner == self){
        for (i = 0; i < buffer->size; i++){
            if(kill(buffer->wallets[i].pid, SIGUSR1) == -1){
                perror("kill");
                exit(EXIT_FAILURE);
            }
        }
    }

    /* Se prepara una estructura temporal para indicar que ya no hay solución. */
    tmp.solution = -1;
    if (write(fd_pipe, &tmp, sizeof(MiningData)) == -1){
        perror("Write pipe ganador");
        exit(EXIT_FAILURE);
    }
}
return 1;
```

Cuando expira el tiempo de vida del proceso o llega una SIGINT, cada minero retira su PID de la lista compartida y, si era el último, marca en la memoria compartida valores defecto que indican el cierre definitivo, envía un bloque de finalización al monitor y libera todos los recursos: semáforos, cola de mensajes, memoria compartida y desmapeo. El registrador, al leer un bloque con solución = −1 o detectar EOF en el pipe, cierra su bucle y finaliza, avisando si algo salió mal. Esta estrategia garantiza una terminación ordenada y la limpieza completa de todos los mecanismos de sincronización.

Además, para mejorar la claridad y el mantenimiento del código se han extraído varias tareas en funciones independientes. La función miningThread() encapsula todo el bucle de búsqueda en un rango concreto y solo devuelve el resultado cuando encuentra la solución, aislando la lógica de hash y sincronización atomizada.

```c
/* Función de minado ejecutada en un hilo.
   Recorre un rango de valores buscando un candidato cuya función de hash coincida con target_value.
   Si lo encuentra, marca la solución encontrada y sale retornando el valor encontrado. */
void *miningThread(void *range){
    long int i;
    Range *r = (Range *) range;

    for (i = r->ini; i < r->end && atomic_load(&flag) == 0 && !got_usr2; i++){
        if (pow_hash(i) == r->target_value){
            atomic_store(&flag, 1);
            pthread_exit((void *)(uintptr_t)i);
        }

    }
    pthread_exit(NULL);
}
```

Con notify_all_except() se agrupa el envío de señales SIGUSR2 a todos los procesos excepto al emisor, evitando duplicar ese bucle en distintos puntos. La función record_vote() factoriza la actualización atómica de los contadores de votos, manejando tanto votos positivos como totales bajo un mismo semáforo.

```c
/* Envía SIGUSR2 a todos (menos al emisor) */
static void notify_all_except(MiningData *minerData, pid_t self, int sig) {
    for (int i = 0; i < minerData->size; i++) {
        if (minerData->wallets[i].pid != self) {
            if (kill(minerData->wallets[i].pid, sig) == -1) {
                perror("kill");
                exit(EXIT_FAILURE);
            }
        }
    }
}

/* Factoriza la actualización de votos (positivo o sólo total) */
static void record_vote(MiningData *minerData, bool positive) {
    sem_wait(&minerData->sem_mutex);
    /* Voto positivo: incrementa votos positivos y totales de forma atómica */
    if (positive) {
        minerData->posVotes++;
    }
    /* Voto negativo: sólo incrementa el total de votos de forma atómica */
    minerData->totalVotes++;
    sem_post(&minerData->sem_mutex);
}
```

Sobre esa base, voteWinner() y voteLoser() implementan de forma limpia las dos caras de la votación: el primero notifica, emite su voto, espera al resto y otorga la recompensa en caso de mayoría; el segundo espera la señal de inicio de votación y registra su voto. Esta modularización facilita futuras extensiones, pruebas unitarias y mejora la legibilidad al separar responsabilidades bien definidas.

```c
void voteWinner(MiningData *minerData, pid_t pid_miner) {
    int index = -1;
    minerData->totalVotes = minerData->posVotes = 0;

    /* Notifica a todos los demás */
    notify_all_except(minerData, pid_miner, SIGUSR2);

    /* Encontrar mi índice para la recompensa */
    for (int i = 0; i < minerData->size; i++) {
        if (minerData->wallets[i].pid == pid_miner) {
            index = i;
            break;
        }
    }

    /* Emitir mi voto */
    bool valid = (pow_hash(minerData->solution) == minerData->target);
    record_vote(minerData, valid);

    /* Espera al resto (o timeout) */
    for (int try = 0; minerData->totalVotes < minerData->size && try < MAX_TRY; try++) {
        usleep(500);
    }

    /* Premio si mayoría valida */
    if (minerData->posVotes >= minerData->totalVotes/2 && index != -1) {
        sem_wait(&minerData->sem_mutex);
        minerData->wallets[index].monedas++;
        sem_post(&minerData->sem_mutex);
    }
}

void voteLoser(MiningData *minerData, const sigset_t sigmask_usr2) {
    /* Si aún no ha llegado la señal SIGUSR2 que inicia la votación,
       espera bloqueando hasta recibirla */
    if (!got_usr2) {
        sigsuspend(&sigmask_usr2);
    }

    /* Reinicia la bandera para futuras rondas */
    got_usr2 = 0;

    /* Comprueba si la solución actual valida el objetivo */
    bool valid = (pow_hash(minerData->solution) == minerData->target);
    record_vote(minerData, valid);
}
```

El proceso **Comprobador** se inicia simplemente ejecutando ./monitor, sin parámetros, y desde el primer momento valida que no se hayan pasado argumentos inesperados, saliendo con un mensaje de error si argc != 1.

```c
/* Este proceso arranca a través de un programa que se debe ejecutar sin parámetros:
    ./ monitor donde monitor es el nombre del ejecutable.*/
if (argc != 1){
    printf("Formato: %s\n", argv[0]);
    exit(EXIT_FAILURE);
}
```

De inmediato crea un segmento de memoria compartida llamado SHM_NAME, lo trunca al tamaño de la estructura circular_buffer y lo mapea en su espacio de direcciones. Con ello reserva espacio para un buffer circular de cinco bloques y tres semáforos anónimos (sem_empty, sem_mutex y sem_fill) que implementan el algoritmo de productor–consumidor. Esta inicialización se abstrae en la función buffer_init(), que devuelve un puntero al buffer listo para usar.

```c
/* El proceso principal, Comprobador, crea e inicializa un segmento de memoria compartida
de monitoreo, dándole tamaño, enlazándolo a su espacio de memoria, y creando los semáforos
oportunos.*/

fd_shm = shm_open(SHM_NAME, O_RDWR | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR);
if (fd_shm == -1) {
    if (errno == EEXIST) {
        fd_shm = shm_open(SHM_NAME, O_RDWR, 0);
        if (fd_shm == -1) {
            perror("Error opening the shared memory segment");
            exit(EXIT_FAILURE);
        }
        else {
            perror("Shared memory segment open\n");
        }
    }
    else {
        perror("Error creating the shared memory segment\n");
        exit(EXIT_FAILURE);
    }
}
else {
    perror("Shared memory segment created\n");
}


/*truncate*/ /*Espacio que quieremos usar en la memoria*/
if (ftruncate(fd_shm, sizeof(circular_buffer)) == -1){
    perror("ftruncate");
    shm_unlink(SHM_NAME);
    exit(EXIT_FAILURE);
}

/* Mapping of the memory segment. */
buffer = mmap(NULL, sizeof(circular_buffer), PROT_READ | PROT_WRITE, MAP_SHARED, fd_shm, 0);
close(fd_shm);
if (buffer == MAP_FAILED) {
    perror("mmap");
    shm_unlink(SHM_NAME);
    exit(EXIT_FAILURE);
}

/* Inicializar buffer */
buffer = buffer_init(buffer);
if (buffer == NULL){
    exit(EXIT_FAILURE);
}
```

A continuación, el Comprobador hace fork() para lanzar el proceso **Monitor** como hijo. El padre permanece en un bucle de recepción de bloques: abre o crea la cola de mensajes MQ_NAME en modo lectura, y espera con mq_receive() a que el Ganador le envíe cada MiningData. Cada bloque recibido se vuelve a validar llamando a la función monitor(target, solution), que encapsula la llamada a pow_hash() y devuelve si la solución efectivamente cumple el objetivo.

De este modo queda claro que la comprobación de integridad está separada de la lógica de recepción.

```c
pid = fork();
if (pid == -1) {
    perror("fork");
    exit(EXIT_FAILURE);
}

block.solution = 0;
block.wallets[0].pid = -1;


/* El proceso Comprobador se encarga de recibir y validar los bloques enviados por el
    proceso Ganador en cada ronda.*/
if (!pid){
    /* Apertura/creación del semáforo */
    sem = sem_open(SEM_NAME, O_RDWR | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR, 0);
    if (sem == SEM_FAILED) {
        if (errno == EEXIST) {
            sem = sem_open(SEM_NAME, O_RDWR, S_IRUSR | S_IWUSR, 0);
            if (sem == SEM_FAILED) {
                perror("sem_open");
                exit(EXIT_FAILURE);
            }
        } else {
            perror("sem_open");
            exit(EXIT_FAILURE);
        }
    }
    sem_post(sem);

    /* Apertura/creación de la cola de mensajes */
    mq = mq_open(MQ_NAME, O_CREAT | O_RDONLY | O_EXCL, S_IRUSR | S_IWUSR, &attributes);
    if (mq == (mqd_t)-1) {
        if (errno == EEXIST) {
            mq = mq_open(MQ_NAME, O_RDONLY, S_IRUSR | S_IWUSR, &attributes);
            if (mq == (mqd_t)-1) {
                perror("mq_open");
                exit(EXIT_FAILURE);
            }
        } else {
            perror("mq_open");
            exit(EXIT_FAILURE);
        }
    }

    i = 0;
    while(block.solution != -1 && block.wallets[0].pid != 0){
        /* Recibe un bloque a través de la cola de mensajes. */
        if (mq_receive(mq, (char *)&m, sizeof(MiningData), NULL) == -1) {
            perror("mq_recieve");
            block.solution = -1; /* Indicar finalización en el bloque */

            /* Insertar el bloque terminador en el buffer compartido */
            sem_wait(&buffer->sem_mutex);
            buffer_addElem(buffer, block);
            sem_post(&buffer->sem_mutex);

            sem_wait(sem);
            sem_close(sem);
            exit(EXIT_FAILURE);
        }
```

```c
        /* Comprueba el bloque de nuevo (con independencia de sus votos) ajustando
            la bandera que indica si es correcto o no.*/

        /* Actualización de los campos del bloque con los datos recibidos */
        block.id = i;
        block.target = m.target;
        block.solution =  m.solution;
        block.winner = m.winner;
        block.posVotes = m.posVotes;
        block.totalVotes = m.totalVotes;
        block.size = m.size;
        memcpy(block.wallets, m.wallets, m.size * sizeof(Wallet));

        /* Verificación y validación de la solución */
        if (block.posVotes >= block.totalVotes / 2){
            block.is_valid_solution = monitor(block.target, block.solution);
        }
```

Una vez validado el bloque, el Comprobador lo inserta en el buffer compartido siguiendo el protocolo de productor: primero hace sem_wait(sem_empty) para asegurar que hay hueco, luego sem_wait(sem_mutex) para zona crítica y llama a buffer_addElem(buffer, block). Tras liberar el mutex y señalizar sem_fill, el bloque queda disponible para el Monitor. Todo este manejo de semáforos y buffer se centraliza en funciones de buffer.c, evitando duplicar código y facilitando posibles cambios en la implementación del buffer.

```
        /* Una vez comprobado, lo introduce en memoria compartida para que lo lea Monitor, y
            vuelve a repetir el proceso de recepción/comprobación/escritura.*/
        sem_wait(&buffer->sem_empty);
        sem_wait(&buffer->sem_mutex);
        buffer_addElem(buffer, block);
        sem_post(&buffer->sem_mutex);
        sem_post(&buffer->sem_fill);

        i++;
    }

    /* El proceso Comprobador finaliza cuando recibe un bloque de finalización a través
        de la cola de mensajes, o cuando le llega la señal SIGINT. En cualquiera de los dos casos, notifica
        el cierre del sistema escribiendo un bloque de finalización en memoria compartida para Monitor.
        Por último, espera la finalización del proceso Monitor, imprime un mensaje de aviso en el caso
        de que no termine con el código de salida EXIT_SUCCESS, y termina liberando todos sus recursos.*/
    sem_wait(sem);
    sem_close(sem);
    mq_close(mq);

    mq_unlink(MQ_NAME);
    shm_unlink(SHM_NAME);

    buffer_destroy(buffer);
    munmap(buffer, sizeof(circular_buffer));

    wait(NULL);
    exit(EXIT_SUCCESS);
```

El proceso **Monitor**, por su parte, arranca justo tras el fork() y entra en un bucle que extrae bloques con el esquema de consumidor: sem_wait(sem_fill), sem_wait(sem_mutex), block = buffer_extractElem(buffer), y luego sem_post(sem_mutex) y sem_post(sem_empty). Si detecta un bloque terminador (solución = –1), rompe el ciclo y libera el mapeo de memoria antes de salir con EXIT_SUCCESS. Mientras tanto, cada bloque se imprime por pantalla en el mismo formato que los Registradores, mostrando ID, ganador, target, solución (validad o rechazada), votos y lista de wallets.

```c
} else {  /* El proceso Monitor arranca como hijo del proceso Comprobador. */

    /* El proceso Monitor se encarga de mostrar los bloques por pantalla*/
    while (1) {
        i = 0;
        /* 1. Extrae un bloque, usando el esquema de productor–consumidor. */
        sem_wait(&buffer->sem_fill);
        sem_wait(&buffer->sem_mutex);
        block = buffer_extractElem(buffer);
        sem_post(&buffer->sem_mutex);
        sem_post(&buffer->sem_empty);

        /* Si se recibe el bloque terminador, se sale inmediatamente */
        if (block.solution == -1 || block.wallets[0].pid == 0)
            break;

        /* 2. Lo muestra por pantalla usando el mismo formato que los procesos Registrador, y repite el
           ciclo de extracción/muestra.*/
        if (block.is_valid_solution) {
            printf("\nBloque %d minado por %d\nSolution : %08ld (accepted)\nVotes : %d/%d\nWallets :",
                block.id, block.winner, block.solution, block.posVotes, block.totalVotes);
            for (int i = 0; i < block.size; i++) {
                printf(" %d:%d", block.wallets[i].pid, block.wallets[i].monedas);
            }
            printf("\n\n");
        } else {
            printf("\nBloque %d minado por %d\nSolution : %08ld (rejected)\nVotes : %d/%d\nWallets :",
                block.id, block.winner, block.target);
            printf("\nSolution : %08ld (rejected)\nVotes : %d/%d\nWallets :",
                block.solution, block.posVotes, block.totalVotes);
        }

        while (i < block.size) {
            printf(" %d:%d", block.wallets[i].pid, block.wallets[i].monedas);
            i++;
        }
        printf("\n\n");
    }

    /* Cuando se extrae el bloque de finalización, libera todos los recursos y termina con
       código de salida EXIT_SUCCESS si todo fue correcto, o EXIT_FAILURE en caso de error.*/
    munmap(buffer, sizeof(circular_buffer));
    exit(EXIT_SUCCESS);
}
return 0;
```

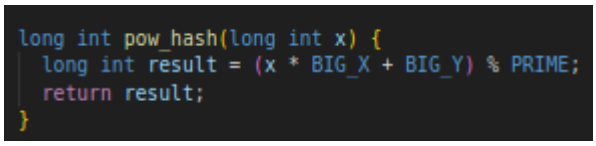
```
int printf(const char *__restrict__ __format, ...)
Write formatted output to stdout.
This function is a possible cancellation point and therefore not marked with __THROW.
```

La finalización tanto de Comprobador como de Monitor se gestiona con un bloque especial de terminación o con SIGINT. Cuando el Comprobador recibe la señal de cierre o detecta el bloque final en la cola, inserta un bloque sentinela en el buffer para despertar al Monitor y luego espera con wait(NULL) a que el hijo termine. Tras comprobar su código de salida, cierra y elimina semáforos, cola de mensajes y segmento compartido, y destruye el buffer con buffer_destroy().

Se usa la función monitor(), que valida si una solución encontrada por un minero es aceptada o no.

```c
/**
 * @brief Valida si una solución encontrada por un minero es aceptada o no.
 * @param target Valor objetivo que se busca alcanzar con la función hash.
 * @param solution Valor encontrado por el miner, cuyo hash será verificado.
 * @return 1 si la solución es válida (hash(solution) == target), 0 en caso contrario.
 */
int monitor(long int target, long int solution) {
    if (pow_hash(solution) == target) {
        return 1;
    }
    else {
        return 0;
    }
}
```

La función pow_hash, definida en **pow.c**, proviene directamente del enunciado y de las prácticas anteriores.



```
long int pow_hash(long int x) {
  long int result = (x * BIG_X + BIG_Y) % PRIME;
  return result;
}
```

En **buffer.c** hemos reaprovechado íntegramente el código del buffer circular desarrollado en la práctica anterior, adaptándolo solo en los detalles necesarios para encajarlo en el nuevo flujo de procesos. La estructura básica, con índices pwrite y pread, tamaño fijo (SIZE) y un array de block, permanece igual, al igual que las funciones buffer_addElem() y buffer_extractElem(), que simplemente avanzan el índice correspondiente con aritmética modular.

```c
circular_buffer *buffer_init(circular_buffer *p_buffer) {
    p_buffer->pwrite = 0;
    p_buffer->pread = 0;
    p_buffer->size = SIZE;

    if (sem_init(&p_buffer -> sem_fill, 1, 0)){
        perror("sem_init");
        free(p_buffer);
        return NULL;
    }

    if (sem_init(&p_buffer -> sem_empty, 1, 5)){
        perror("sem_init");
        sem_close(&p_buffer -> sem_fill);
        free(p_buffer);
        return NULL;
    }

    if (sem_init(&p_buffer -> sem_mutex, 1, 1)){
        perror("sem_init");
        sem_close(&p_buffer -> sem_fill);
        sem_close(&p_buffer -> sem_empty);
        free(p_buffer);
        return NULL;
    }

    return p_buffer;
}

void buffer_destroy(circular_buffer *p_buffer) {
    if (p_buffer == NULL)
        return;

    sem_destroy(&p_buffer -> sem_fill);
    sem_destroy(&p_buffer -> sem_empty);
    sem_destroy(&p_buffer -> sem_mutex);
}


void buffer_addElem(circular_buffer *p_buffer, block bloq) {
    p_buffer -> bloques[p_buffer -> pwrite] = bloq;
    p_buffer -> pwrite ++;
    p_buffer -> pwrite %= p_buffer -> size;
}

block buffer_extractElem(circular_buffer *p_buffer) {
    block elem;

    elem = p_buffer -> bloques[p_buffer -> pread];
    p_buffer -> pread ++;
    p_buffer -> pread %= p_buffer->size;

    return elem;
}
```

Donde sí hubo que ajustar fue en la inicialización de semáforos. Por ello en buffer_init() usamos sem_init(&sem, 1, valor) con el segundo parámetro a 1 (pshared) en lugar de 0, y establecemos los contadores en 0 para sem_fill, en el tamaño máximo (5) para sem_empty y en 1 para sem_mutex. La función buffer_destroy() se encarga únicamente de sem_destroy() sobre esos semáforos, confiando en que la liberación del segmento compartido y el munmap pertenezcan a la lógica de terminación del proceso Comprobador.

Además, para organizar y aislar la lógica de gestión de la memoria compartida de mineros, se ha introducido el archivo **shm_miner.c**, donde se agrupan las funciones que inicializan, destruyen y modifican la estructura compartida MiningData. La función initMinerData() recibe el puntero mapeado al segmento de memoria compartida y se encarga de poner a sus campos indicadores de estado (ganador, solución y contadores de votos) en valores sentinela (–1 o 0), y de crear un semáforo anónimo sem_mutex en modo procesalmente compartido. Su gemela, destroyMinerData(), simplemente invoca sem_destroy() para liberar dicho semáforo cuando ya no es necesario.

```c
MiningData *initMinerData(MiningData *minerData) {
    if (minerData == NULL)
        return NULL;

    minerData->winner = -1;
    minerData->solution = -1;
    minerData->totalVotes = 0;
    minerData->posVotes = 0;
    minerData->size = 0;

    if (sem_init(&minerData->sem_mutex, 1, 1) != 0) {
        perror("sem_init");
        free(minerData);
        return NULL;
    }
    return minerData;
}

void destroyMinerData(MiningData *minerData) {
    if (minerData == NULL)
        return;

    sem_destroy(&minerData->sem_mutex);
}

int addMiner(MiningData *minerData, pid_t pid) {
    if (minerData->size >= MAX_MINER) {
        return -1;
    }
    minerData->wallets[minerData->size].pid = pid;
    minerData->size++;
    return 0;
}

void removeMiner(MiningData *minerData, pid_t pid) {
    int i;
    for (i = 0; i < minerData->size; i++) {
        if (minerData->wallets[i].pid == pid) {
            memmove(&minerData->wallets[i], &minerData->wallets[i + 1], (minerData->size - i - 1) * sizeof(Wallet));
            minerData->size--;
            return;
        }
    }
}
```

Para manejar dinámicamente la lista de mineros registrados, addMiner() añade el PID de un nuevo minero al siguiente hueco disponible del array wallets[], incrementando el contador size siempre que no se haya alcanzado MAX_MINER. En sentido inverso, removeMiner() busca el PID indicado y, usando memmove(), comprime el array sobre la posición eliminada y decrementa size, garantizando así que no queden “huecos” ni IDs huérfanos.

Para gestionar de forma ordenada toda la información compartida entre mineros, monitor y registrador, hemos definido varias estructuras clave en un único punto de cabecera. La estructura Wallet agrupa el PID de cada minero y las monedas que acumula, mientras que MiningData amplía esta información con semáforo de exclusión mutua, el desafío actual (target y solution), indicadores de validez y los contadores de votos, junto al array de wallets. Para dividir el trabajo interno de cada hilo de minería, Range especifica un subrango (ini–end) y el valor objetivo que debe buscarse. Cada bloque completo de la cadena se representa con block, que incluye un identificador, los mismos campos de validación y wallets que MiningData, y es el tipo que circula por el buffer. Este buffer, descrito en circular_buffer, incorpora índices de

lectura/escritura, tres semáforos (sem_empty, sem_fill, sem_mutex) y un array fijo de bloques de longitud SIZE, permitiendo un productor–consumidor robusto entre Comprobador y Monitor. Además, hemos declarado los atributos de la cola de mensajes (mq_attr) para ajustarla a nuestro MiningData, y definido macros (REG_FILE, SHM_MINER, SHM_NAME, MQ_NAME, SEM_NAME) para unificar nombres de recursos compartidos en todo el proyecto.

c) Detallar las limitaciones que tiene el sistema, así como las pruebas realizadas para comprobar que el sistema es correcto. Si no se han implementado todos los requisitos, o el sistema presenta errores conocidos, es importante especificarlo en este apartado

- Se ejecuta el sistema con dos mineros casi simultáneos y un monitor:

```
diego-gomez@diego-gomez-MS-7C95:~/Escritorio/Soper_proy$ ./monitor & ./miner 2 1 & ./miner 3 1
[1] 20806
[2] 20807
Shared memory segment created
: Success
Shared memory segment created
: Success
Shared memory segment open
: File exists
Id : 000
Winner : 20807
Target : 00000000
Solution : 08916551 (validated)
Votes : 1/1
Wallets : 20807:1 20808:0

Id : 001
Winner : 20808
Target : 08916551
Solution : 00000000 (rejected)
Votes : 0/1
Wallets : 20807:1 20808:1

Id : 002
Winner : 20808
Target : 00000000
Solution : 08916551 (validated)
Votes : 2/2
Wallets : 20807:1 20808:2
```

...

```
Id : 120
Winner : 20808
Target : 08343206
Solution : 06079329 (validated)
Votes : 1/1
Wallets : 20808:91

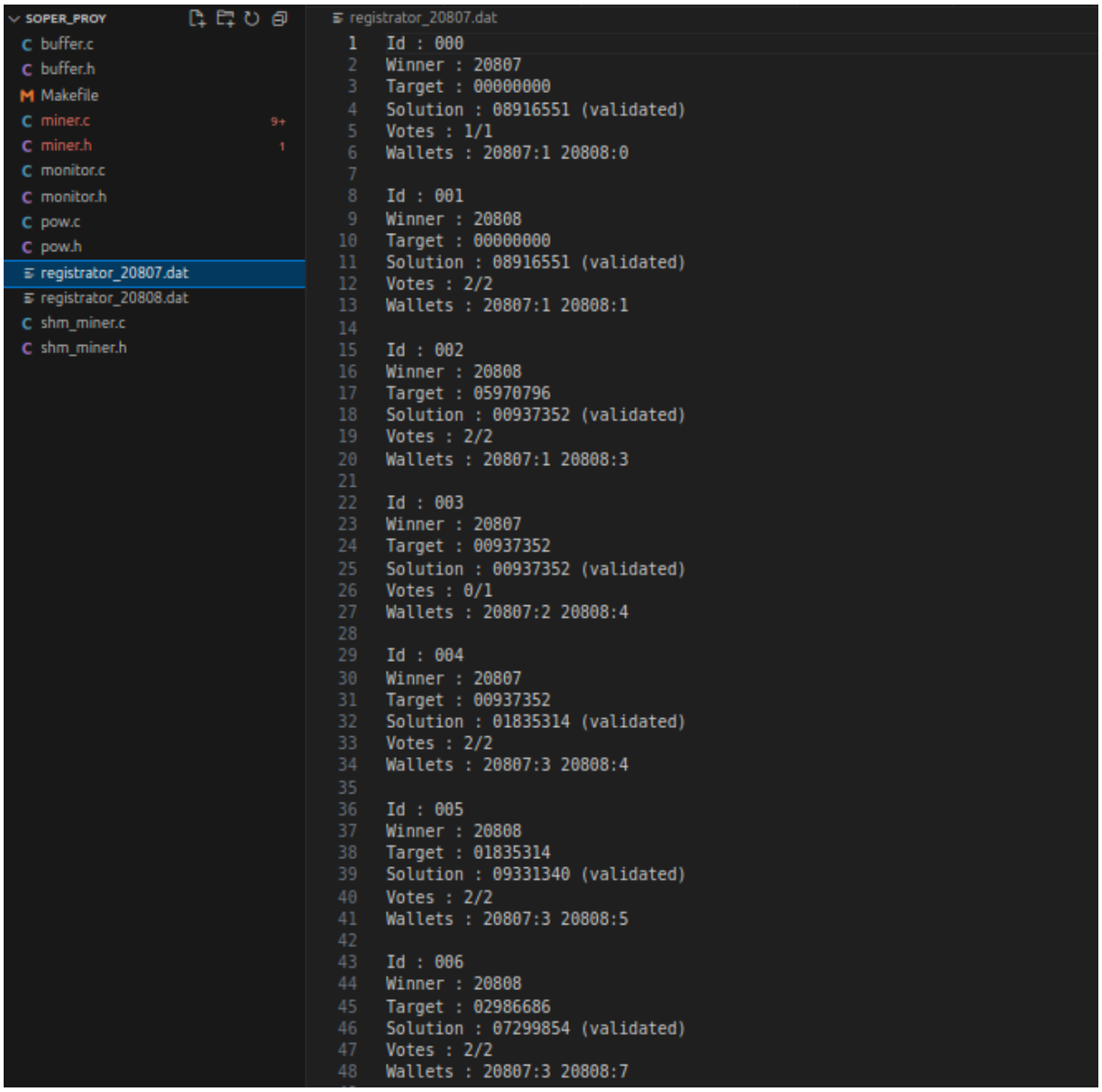
Id : 121
Winner : 20808
Target : 06079329
Solution : 01490448 (validated)
Votes : 1/1
Wallets : 20808:92

Id : 122
Winner : 20808
Target : 01490448
Solution : 09834408 (validated)
Votes : 1/1
Wallets : 20808:93

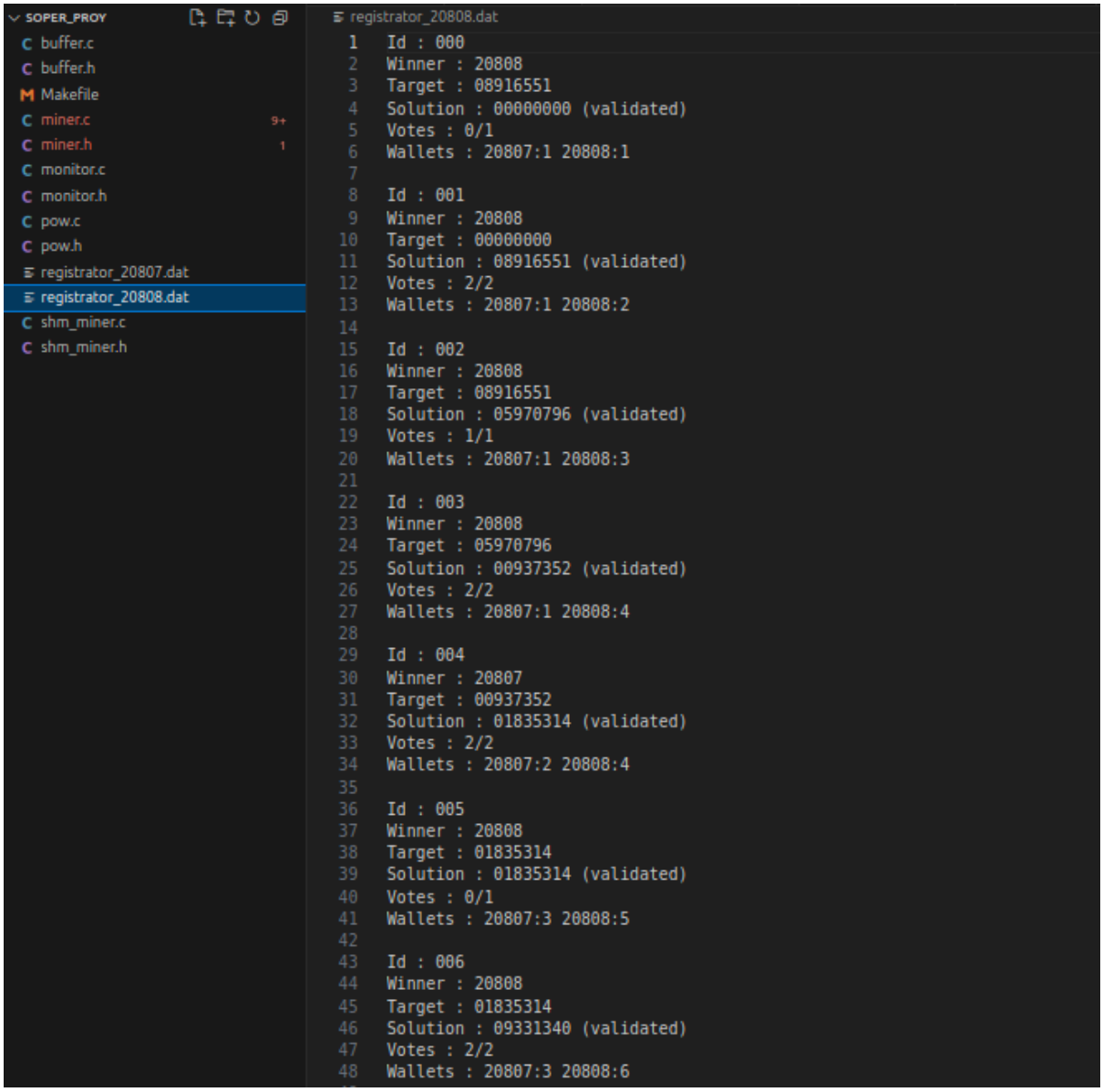
Id : 123
Winner : 20808
Target : 09834408
Solution : 08968456 (validated)
Votes : 1/1
Wallets : 20808:94

[1]-  Hecho                 ./monitor
[2]+  Hecho                 ./miner 2 1
```

registrator_20807.dat:



registrator_20808.dat:


SOPER_PROY
buffer.c
buffer.h
Makefile
miner.c 9+
miner.h 1
monitor.c
monitor.h
pow.c
pow.h
registrator_20807.dat
registrator_20808.dat
shm_miner.c
shm_miner.h
registrator_20808.dat
Id : 000
Winner : 20808
Target : 08916551
Solution : 00000000 (validated)
Votes : 0/1
Wallets : 20807:1 20808:1
Id : 001
Winner : 20808
Target : 00000000
Solution : 08916551 (validated)
Votes : 2/2
Wallets : 20807:1 20808:2
Id : 002
Winner : 20808
Target : 08916551
Solution : 05970796 (validated)
Votes : 1/1
Wallets : 20807:1 20808:3
Id : 003
Winner : 20808
Target : 05970796
Solution : 00937352 (validated)
Votes : 2/2
Wallets : 20807:1 20808:4
Id : 004
Winner : 20807
Target : 00937352
Solution : 01835314 (validated)
Votes : 2/2
Wallets : 20807:2 20808:4
Id : 005
Winner : 20808
Target : 01835314
Solution : 01835314 (validated)
Votes : 0/1
Wallets : 20807:3 20808:5
Id : 006
Winner : 20808
Target : 01835314
Solution : 09331340 (validated)
Votes : 2/2
Wallets : 20807:3 20808:6

- Se arranca un minero y el monitor, y un segundo más tarde se arranca otro minero, que se une al sistema

```
diego-gomez@diego-gomez-MS-7C95:~/Escritorio/Soper_proy$ ./monitor & ./miner 2 1 & ( sleep 1 ; ./miner 1 3)
[1] 30360
[2] 30361
Shared memory segment created
: Success
Shared memory segment created
: Success
Id : 000
Winner : 30361
Target : 00000000
Solution : 08916551 (validated)
Votes : 1/1
Wallets : 30361:1

Id : 001
Winner : 30361
Target : 08916551
Solution : 05970796 (validated)
Votes : 1/1
Wallets : 30361:2

Id : 002
Winner : 30361
Target : 05970796
Solution : 00937352 (validated)
Votes : 1/1
Wallets : 30361:3
```

...

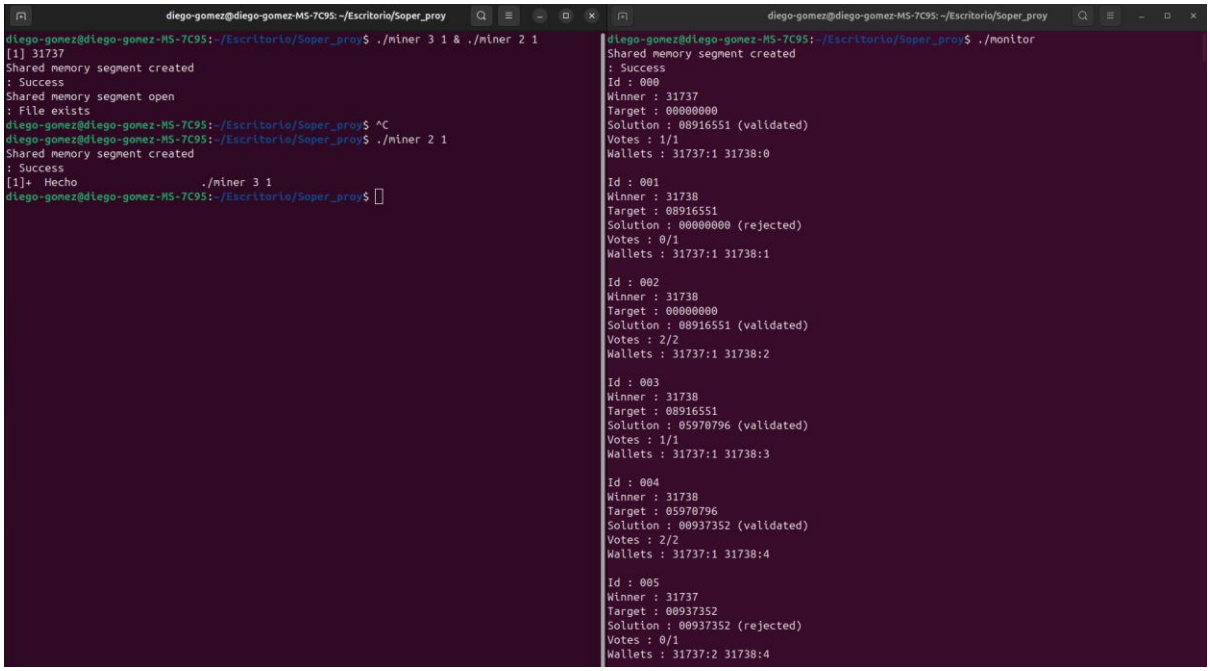
```
Id : 107
Winner : 30361
Target : 02000011
Solution : 09327418 (validated)
Votes : 2/2
Wallets : 30361:90 30362:18

Id : 108
Winner : 30362
Target : 09327418
Solution : 09327418 (validated)
Votes : 0/2
Wallets : 30362:18

Id : 109
Winner : 30362
Target : 09327418
Solution : 08405727 (validated)
Votes : 1/1
Wallets : 30362:19

[1]-  Hecho                   ./monitor
[2]+  Hecho                   ./miner 2 1
```

- Se ejecuta primero el monitor, luego se lanza el sistema con dos mineros casi simultáneos, se cierra uno de ellos, y se añade otro más.



...

